

Experiment 5:

DC to AC Inverter

ECEN 5517-012

Khalid Shahba

Stewart Rojec

BENCH 5

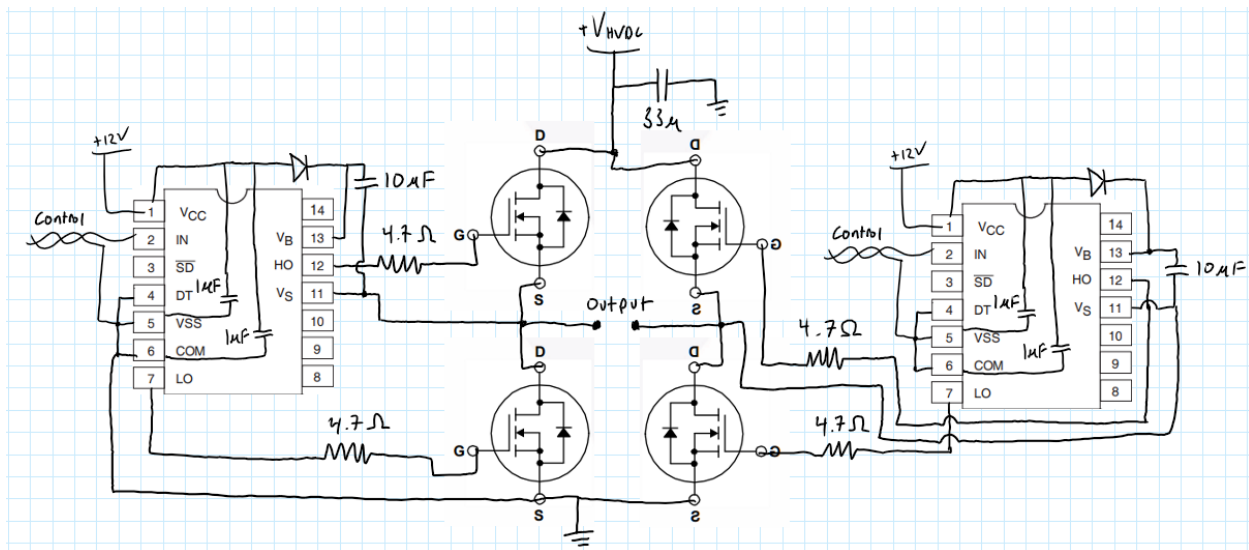
I. Introduction

The goal of this experiment was to convert the HVDC of the closed-loop cascaded buck converter into 120 Vrms 60 Hz AC Power. It used an H-Bridge inverter driven by a built-in dead-time half-bridge driver. First, a modified sine wave is created using a complementary phase-delayed PWM on each half-bridge driver. A true sine wave inverter is also created by implementing an LC filter and modifying the PWM generating code to vary duty cycle in a sine wave pattern.

II. Experimental Procedure and Results

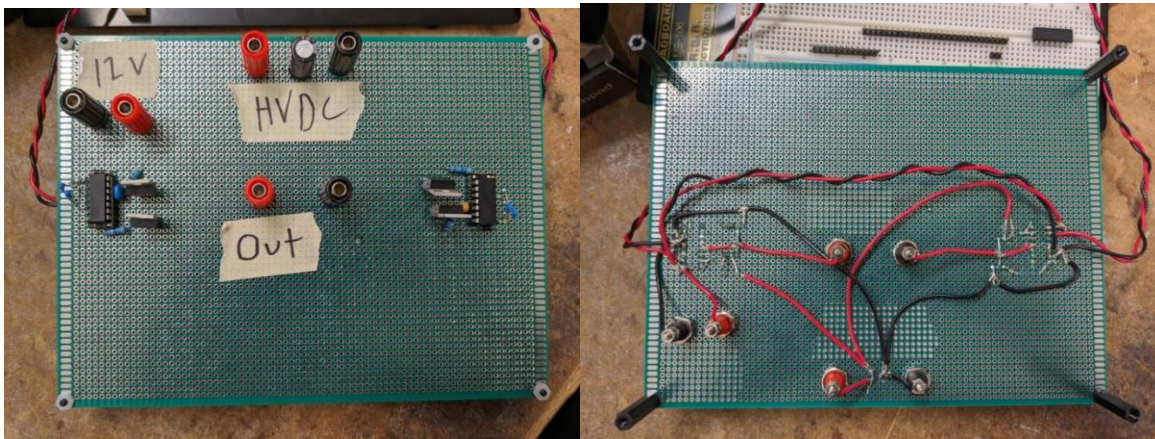
a. Modified Sine-Wave Inverter Circuit Diagram

- i. Complete Labeled Circuit Diagram of the Modified Sine-Wave Inverter:



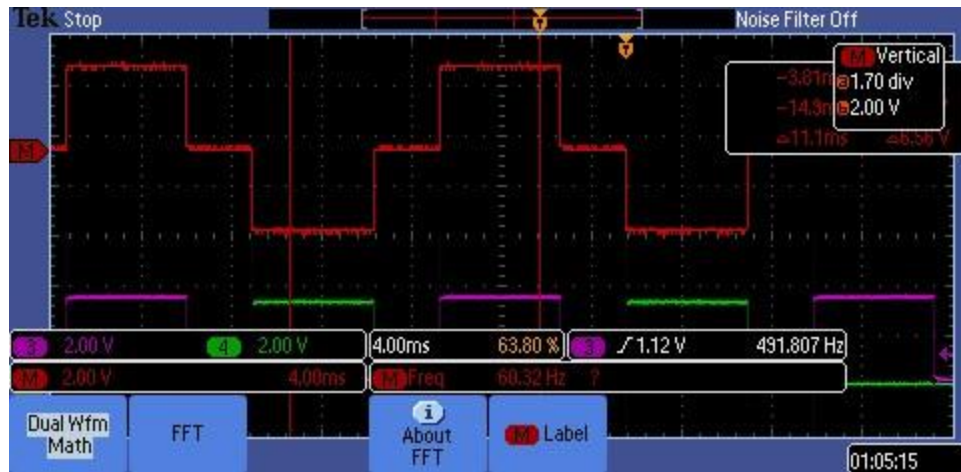
b. Modified Sine-Wave Inverter Circuitry

- i. Constructed Inverter Photos:



c. Modified Sine-Wave Gate Control Signals

- i. Control Signal Oscilloscope Capture:



Purple: Q1/Q2 IO (J6.5), **Green:** Q3/Q4 IO (J2.8), **Red:** “Purple – Green” showing a preview of the load voltage just divided down.

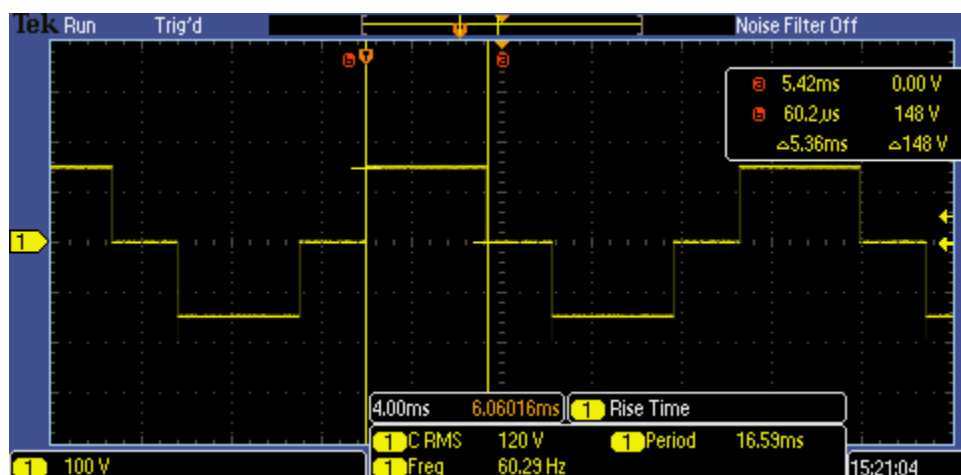
d. Measured Duty-Cycle, Input DC Voltage, Output RMS Voltage, Output RMS Current

- Measured Duty-Cycle, Input DC Voltage, Output RMS Voltage, Output RMS Current

Measured Input DC Voltage	Output RMS Voltage	Output RMS Current	Measured Duty Cycle
150V	120.2V	0.296A	0.647

Measured frequency was **60.19 Hz**.

- Measured Output Voltage Waveform



Yellow: Output Voltage Across Load

e. True Sine-Wave Inverter

- Documented Inductor Design Procedure and Inductance Values

Our inductor values were chosen rather haphazardly. We began designing for a cutoff frequency of 1000 Hz, were told that was excessive by TAs, tested for a cutoff of 20 kHz and didn't like our ripple performance. We thus went to LTSpice and just optimized. Our optimization criterion was

that we wanted the largest inductors possible before we reached a point of diminishing returns where we couldn't reduce inductor current past the 60 Hz current component peak. Any more inductance past that point would've just contributed to getting closer to the saturation of the inductor. Using LTSpice, we tested with 5mH inductors on both sides of the load and got a response where any higher inductance didn't change the peak inductor current by much and called it good enough. The peak inductor current at this point was roughly 700 mA but in our design we used a factor of safety and designed for a maximum current 1A.

Since we already start with a simulated desired inductance of 5mH, we begin via the Kg method.

$$K_g \geq \frac{\rho L^2 I_{max}^2}{B_{max}^2 R K_u} 10^8 \text{ cm}^5$$

For a ρ of $1.724 \times 10^{-6} \Omega \text{cm}$, an $L = 5 \text{mH}$, I_{max} of 1A, B_{max} of 0.3T, and maximum allowable series resistance of 0.5Ω with $K_u = 0.5$ nominally, we reach a K_g of 0.1916 cm^5 . Thus, our stocked PQ26/25 will be too small to accommodate for these specifications as it has a K_g of 0.125 cm^5 . Thus, we must go for the PQ32/20 core with $K_g = 0.203 \text{ cm}^5$ (which is good because we ran out of PQ26/25 already). Using that core, the number of turns to make a 5mH inductor is:

$$n = \frac{L I_{max}}{B_{max} A_c} 10^4$$

Thus, **98 turns** are required to reach 5mH.

The air gap length is determined as follows:

$$l_g = \frac{\mu_o A_c n^2}{L} 10^{-4} \text{ m}$$

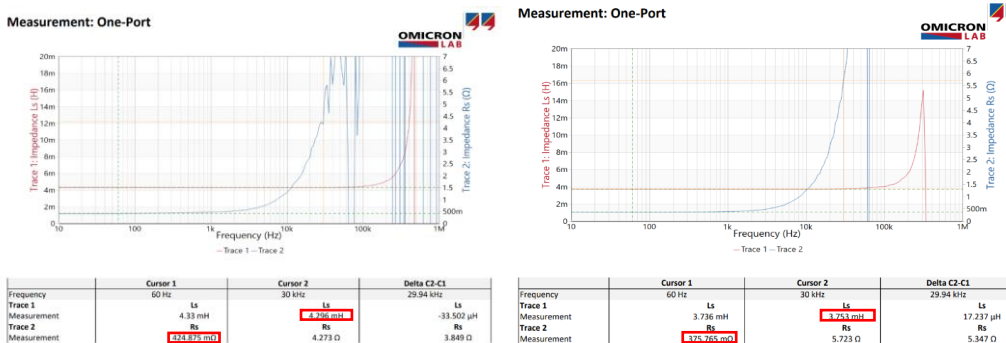
Thus, **16 mils** of total airgap is required. This will be done through 2 8-mil shims on either side of the core.

Finally, we check to see which maximum wire gauge to use.

$$A_w \leq \frac{K_u W_A}{n} \text{ cm}^2$$

Thus, we had to pick 26 AWG as our maximum allowable area was $2.4 \times 10^{-3} \text{ cm}^2$. We went with **23 AWG** anyway and just acknowledged we would probably get fewer turns and less inductance anyway.

As predicted, we ended up with actual measured inductor values of **4.296 mH** and **3.71 mH**. We ran an LTSpice validation on these values to ensure that the peak current hadn't risen too much and it behaved relatively the same way.



ii. C Code:

```
/*
 * PWM_ADC2.c
 *
 * Created on: Feb 8, 2022
 * Author: Aakash Kamalapur
 *
 * ECEN 4517/5517 Experiment #3: open-loop modulation of Buck converter
 *
 * Updates:
 * 02/06/2024: bug fixed in adc_isr, load sensed_voltage from ADCRESULT1 and sensed_current from ADCRESULT0 (L. Corradini)
 */
```

```

#include "DSP28x_Project.h"

void InitEPwm1Example(unsigned int, unsigned int); // PWM outputs on J6.1 and J6.2 of the F28027 LaunchPad
void InitADC_sampling(void); // ADC input on J5.3 of the F28027 LaunchPad
void update_duty(unsigned int); // Update PWM duty cycle
interrupt void adc_isr(void); // ADC interrupt service routine invoked after end of conversion in each PWM period

unsigned int sensed_voltage,sensed_current; // 12-bit ADC output (0-to-4095 corresponds to 0-to-3.3 V)
unsigned int pulse_width; // PWM pulse width expressed as a count of (1/60 MHz) clock cycles
unsigned int period; // PWM period expressed as a count of (1/60 MHz) clock cycles

unsigned int half_duty; // PWM pulse width expressed as a count of (1/7.5 MHz) clock cycles. Check line # 243
and # 280 for why 7.5 MHz and not 60 MHz
unsigned int mod_period; // PWM period expressed as a count of (1/7.5 MHz) clock cycles. Check line # 243 and
# 280 for why 7.5 MHz and not 60 MHz
unsigned int delay_shift;

volatile unsigned long adc_isr_ctr = 0;
volatile unsigned int i,j;
double Ibatt;
double Vbatt;
double Pbatt;
double Ibatt_max;
double Pbatt_max;
unsigned int D_max;
double avg;
double Ibatt_p = 0;
double Vbatt_p = 0;
double Pbatt_p = 0;

unsigned int duty[500] =
{500,506,512,518,524,530,536,542,548,554,560,565,571,577,583,589,595,601,607,612,618,624,630,635,641,647,652,658,664,669,675,680,686,691,697,
702,708,713,718,724,729,734,739,744,749,755,760,765,769,774,779,784,789,793,798,803,807,812,816,821,825,829,834,838,842,846,850,854,858,862,8
66,870,873,877,881,884,888,891,895,898,901,904,907,910,913,916,919,922,925,927,930,932,935,937,939,942,944,946,948,950,952,954,955,957,959,96
0,962,963,964,965,967,968,969,970,970,971,972,973,973,974,974,974,974,975,975,975,975,975,975,975,974,974,974,973,973,972,971,970,970,969,968,967
,965,964,963,962,960,959,957,955,954,952,950,948,946,944,942,939,937,935,932,930,927,925,922,919,916,913,910,907,904,901,898,895,891,888,884,
881,877,873,870,866,862,858,854,850,846,842,838,834,829,825,821,816,812,807,803,798,793,789,784,779,774,769,765,760,755,749,744,739,734,729,7
24,718,713,708,702,697,691,686,680,675,669,664,658,652,647,641,635,630,624,618,612,607,601,595,589,583,577,571,565,560,554,548,542,536,530,52
4,518,512,506,500,494,488,482,476,470,464,458,452,446,440,435,429,423,417,411,405,399,393,388,382,376,370,365,359,353,348,342,336,331,325,320
,314,309,303,298,292,287,282,276,271,266,261,256,251,245,240,235,231,226,221,216,211,207,202,197,193,188,184,179,175,171,166,162,158,154,150,
146,142,138,134,130,127,123,119,116,112,109,105,102,99,96,93,90,87,84,81,78,75,73,70,68,65,63,61,58,56,54,52,50,48,46,45,43,41,40,38,37,36,35
,33,32,31,30,30,29,28,27,27,26,26,26,25,25,25,25,25,25,26,26,26,27,27,28,29,30,30,31,32,33,35,36,37,38,40,41,43,45,46,48,50,52,54,56,58,61
,63,65,68,70,73,75,78,81,84,87,90,93,96,99,102,105,109,112,116,119,123,127,130,134,138,142,146,150,154,158,162,166,171,175,179,184,188,193,19
7,202,207,211,216,221,226,231,235,240,245,251,256,261,266,271,276,282,287,292,298,303,309,314,320,325,331,336,342,348,353,359,365,370,376,382
,388,393,399,405,411,417,423,429,435,440,446,452,458,464,470,476,482,488,494};

// These are defined by the linker (see F28027.cmd)
extern unsigned int RamfuncsLoadStart;
extern unsigned int RamfuncsLoadEnd;
extern unsigned int RamfuncsRunStart;

// Functions that will be run from RAM need to be assigned to
// a different section. This section will then be mapped using
// the linker .cmd file.
#pragma CODE_SECTION(main, "ramfuncs");
#pragma CODE_SECTION(adc_isr, "ramfuncs");

void main(void)
{
// ***** Beginning of Setup *****

// Step 1. Initialize System Control:
// PLL, WatchDog, enable Peripheral Clocks
// This example function is found in the DSP2802x_SysCtrl.c file.
InitSysCtrl();

// Step 2. Clear all interrupts and initialize PIE vector table:
// Disable CPU interrupts
DINT;

// Initialize the PIE control registers to their default state.
InitPieCtrl();

// Disable CPU interrupts and clear all CPU interrupt flags:
IER = 0x0000;
IFR = 0x0000;

// Initialize the PIE vector table with pointers to the shell Interrupt
// Service Routines (ISR).
InitPieVectTable();

// Step 3. Copy program code from FLASH to RAM and initialize FLASH:
// Copy time critical code and Flash setup code to RAM
// The RamfuncsLoadStart, RamfuncsLoadEnd, and RamfuncsRunStart
// symbols are created by the linker. Refer to the F28027.cmd file.
MemCopy(&RamfuncsLoadStart, &RamfuncsLoadEnd, &RamfuncsRunStart);

// Call Flash Initialization to setup flash waitstates
// This function must reside in RAM
InitFlash();

// Step 4. Initialize all the Device Interrupts:
// ISR functions found within this file.
EALLOW; // This is needed to write to EALLOW protected registers
PieVectTable.ADCINT1 = &adc_isr; // Initialize adc_isr interrupt on ADCINT1
EDIS; // This is needed to disable write to EALLOW protected registers

// Step 5. Initialize all the Device Peripherals:
EALLOW;
InitAdc();
EDIS;

```

```

// Step 6. Initialize GPIO as EPWM1A/B and setup ADC to trigger on EPWM1A TBCTR=period:
EALLOW;
SysCtrlRegs.PCLKCR0.bit.TBCLKSYNC = 0;
EDIS;

InitEPwm1Gpio();           // Sets up the GPIO as PWM pins
InitEPwm3Gpio();           // Sets up the GPIO as PWM pins
InitEPwm4Gpio();           // Sets up the GPIO as PWM pins
InitEPwm1Example(period, 0); // PWM setup with pulse width set to zero
InitEPwm3_4(mod_period, 0); // PWM setup with pulse width set to 50% duty
InitADC_sampling(); // ADC setup

EALLOW;
SysCtrlRegs.PCLKCR0.bit.TBCLKSYNC = 1;
EDIS;

period = 1000; // PWM period set as the number of (1/60 MHz) clock cycles -> Changed to 500 clock cycles to one period for 120 kHz
mod_period = 1000*2; // PWM period for EPWM3 and EPWM 4 set as the number of (1/7.5 MHz) clock cycles. Check line # 243 and # 280 for why
7.5 MHz and not 60 MHz
//half_duty = (int) 31250.0 * D; // PWM duty on EPWM3 AND EPWM4 to setup 120 Hz square wave signal.
//delay_shift = 31250; //+ (int) 31250.0 * (1.0-D);

// Step 7. Example specific code, enable interrupts:
PieCtrlRegs.PIEIER1.bit.INTx1 = 1; // Enable INT 1.1 in the PIE
IER |= M_INT1; // Enable CPU Interrupt 1

// Enable Global Interrupts and higher priority real-time debug events:
EINT; // Enable Global interrupt INTM
ERTM; // Enable Global realtime interrupt DBGEM

// ***** End of Setup *****

// User code
pulse_width = 0; // PWM duty cycle is D = pulse_width/period
D_max = 0;
update_duty(pulse_width);
double RF1 = 6520.0; //custom measured resistor divider values and other constants
double RF2 = 3226.0;
double RS1 = 20000.0;
double RS2 = 6520.0;
double ADC_factor = 3.3/4095.0;
double Rsense = 0.02;
Ibatt = 0; // Ibatt, Vbatt and Pbatt are all initialized to 0 as they are counter variables in our average scheme
Vbatt = 0;
Pbatt = 0;
adc_isr_ctr = 0; // this is the variable counting how many times we enter the ADC ISR (1 time every 10 us).
i = 0;
j = 0;
avg = 0;
unsigned int ctrl_save_time = 0;
while (adc_isr_ctr < 11700){ // 195 ms delay once we initialize and have D = 0.
while (adc_isr_ctr >= 11700 && adc_isr_ctr < 12000)
{
Ibatt += sensed_current*ADC_factor*(RF2+RF1)/RF2 * (1/(Rsense*50.0)); //for the final 5 ms of our 200ms delay, we take averages of the
measured Ibatt and Vbatt
Vbatt += sensed_voltage*ADC_factor*(RS2+RS1)/RS2;
avg++;
}
Ibatt_p = Ibatt/avg; //at the end of summing up hundreds of Ibatts and Vbatts and counting up exactly how much "hundreds" is, we can
create
Vbatt_p = Vbatt/avg; //solid data points that refresh every 200ms in the form of Ibatt_p, Vbatt_p and Pbatt_p.
Pbatt_p = Ibatt_p*Vbatt_p;

Pbatt_max = Pbatt_p; //right away, set this first measured value as the max to beat.
unsigned int n = 0; //this variable is how we can perturb and observe while still keeping track of adc_isr_ctr reaching 60s.
unsigned int c = 0; //this variable keeps track of the direction we went in the previous perturbation.
adc_isr_ctr = 7000000; // we set the adc_isr_ctr manually high to force a sweep when the code first loads.
while (1)
{
// do a general sweep of duty cycle every minute from 0 to 90 (pulse_width of 0 to 540) as a starting point
if (adc_isr_ctr >= 3600000)
{
n = 0;
Pbatt_max = 0;
for(pulse_width = 0; pulse_width < 950; pulse_width += 50){
update_duty(pulse_width);
adc_isr_ctr = 0;
Ibatt = 0;
Vbatt = 0;
while(adc_isr_ctr < 11700){
avg = 0;
while(adc_isr_ctr >= 11700 && adc_isr_ctr < 12000){
Ibatt += sensed_current*ADC_factor*(RF2+RF1)/RF2 * (1/(Rsense*50.0)); // see commented lines 125-142 for the explanation of
Vbatt += sensed_voltage*ADC_factor*(RS2+RS1)/RS2; // this averaging technique
avg++;
}
Ibatt_p = Ibatt/avg;
Vbatt_p = Vbatt/avg;
Pbatt_p = Ibatt_p*Vbatt_p;
if(Pbatt_p > Pbatt_max){ // at the end of each step in the sweep, reassign a
Pbatt_max = Pbatt_p; // MPPT Duty Cycle (D_max) every time a new peak
}
D_max = pulse_width; // power is reached
}
if(pulse_width > 850){ // at the end of the sweep set ADC_isr_ctr back to 0.
adc_isr_ctr = 0;
}
}
}
// now we have the peak D that gets cycled every minute and we perturb and observe
// until 60 seconds reached

```

```

        c = 0; //initially start at a decrease from the D_max arrived at in rough sweep.
        pulse_width = D_max-3;
        update_duty(pulse_width);
    }
    if(D_max >= 950)                // this is a safety feature that says if somehow, D_max has become greater than 95%, restart the sweep
    {                                // by resetting adc_isr_ctr to 600000.
        adc_isr_ctr = 600000;
        D_max = 0;
        pulse_width = 0;
    }
    n++;                            //now that the first perturbation has occured, increment n.
    Ibatt = 0;
    Vbatt = 0;
    avg = 0;
    adc_isr_ctr = (n-1)*12000.0 - 300.0; //manually reset adc_isr_ctr to allow an even transient for all perturbations and observations
    while(adc_isr_ctr < n*12000.0 - 300.0){}
    while(adc_isr_ctr >= n*12000.0 - 300.0 && adc_isr_ctr < n*12000.0){
        Ibatt += sensed_current*ADC_factor*(RF2+RF1)/RF2 * (1/(Rsense*50.0)); // again, use the final 5ms to average hundreds of
measurements
        Vbatt += sensed_voltage*ADC_factor*(RS2+RS1)/RS2;
        avg ++;
    }
    Ibatt_p = Ibatt/avg;
    Vbatt_p = Vbatt/avg;
    Pbatt_p = Ibatt_p*Vbatt_p;
    if(Pbatt_p > Pbatt_max && c == 0) // if we decreased DC and went up in power, do it again. And reset Pbatt_max as always.
    {
        D_max = pulse_width - 1;
        pulse_width = D_max;
        c = 0;
        Pbatt_max = Pbatt_p;
        update_duty(pulse_width);
    }
    else if(Pbatt_p < Pbatt_max && c == 0) // if we decreased DC and went down in power, go up in duty cycle.
    {
        D_max = pulse_width + 1;
        pulse_width = D_max;
        c = 1;
        Pbatt_max = Pbatt_p;
        update_duty(pulse_width);
    }
    else if(Pbatt_p > Pbatt_max && c == 1) // if we increased DC and went up in power, do it again.
    {
        D_max = pulse_width + 1;
        pulse_width = D_max;
        c = 1;
        Pbatt_max = Pbatt_p;
        update_duty(pulse_width);
    }
    else if(Pbatt_p < Pbatt_max && c == 1) // if we increased DC and went down in power, go down in duty cycle.
    {
        D_max = pulse_width - 1;
        pulse_width = D_max;
        c = 0;
        Pbatt_max = Pbatt_p;
        update_duty(pulse_width);
    }
    else                                // if by some rarity, the measured power is identical to the previous one, change nothing
and maintain
    {
        Pbatt_max = Pbatt_p;
    }
    while(Vbatt_p > 13){               // here is our charge control check at the end of every perturbation and observation.
        pulse_width = 0;
        update_duty(pulse_width);
        ctrl_save_time = adc_isr_ctr; //after turning off duty cycle, wait for 200ms before determining if staying is correct
        while(adc_isr_ctr - ctrl_save_time < 120000){}
        Vbatt_p = sensed_voltage*ADC_factor*(RS2+RS1)/RS2 + 1.3; //add a correction factor that incorporates the loss of charging
current added voltage
        pulse_width = 0;
        update_duty(pulse_width);
    }
}

// Update duty cycle by updating the pulse_width for the PWM output on J6.1 of the F28027 LaunchPad
void update_duty(unsigned int pulse_width)
{
    EALLOW;
    EPwm1Regs.CMPA.half.CMPA = pulse_width;
    EDIS;
}

// ADC interrupt service routine, update sensed_voltage
interrupt void adc_isr(void)
{
    sensed_voltage = AdcResult.ADCRESULT1; // Voltage sensed on pin J5.3 of the F28027 LaunchPad, 0-to-(2^12 -1) corresponds to 0-to-3.3 V
    sensed_current = AdcResult.ADCRESULT0; // Voltage sensed on pin J5.4 of the F28027 LaunchPad, 0-to-(2^12 -1) corresponds to 0-to-3.3 V

    update_duty(pulse_width); //Changes the duty cycle of the PWM output signal on pin J6.1

    EALLOW;
    EPwm2Regs.CMPA.half.CMPA = (period)>>1; // Stars the sampling of the current and voltage signals at 0.5*(D.Ts) for every succeeding
switching cycle to sample average values and avoid ringing.
    EDIS; // This could be changed to whatever the student needs based on duration of ringing in their system

    AdcRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; // Clear ADCINT1 flag reinitialize for next SOC
    PieCtrlRegs.PIEACK.all = PIEACK_GROUP1; // Acknowledge interrupt to PIE

    adc_isr_ctr++;

```

```

i = i+1;
if(i > 1){
    if(j >= 500)
    {
        j = 0;
    }
    else
    {
        EPwm3Regs.CMPA.half.CMPA = 2*duty[j];
        EPwm3Regs.CMPB = 2*duty[j];
        j = j+1;
    }
    i = 0;
}
return;
}

// ADC setup with ADC inputs on J5.3 and J5.4 of the F28027 LaunchPad
void InitADC_sampling()
{
    EALLOW;
    AdcRegs.ADCCTL1.bit.INTPULSEPOS = 1; //ADCINT1 trips after AdcResults latch
    AdcRegs.INTSELIN2.bit.INT1E = 1; //Enabled ADCINT1
    AdcRegs.INTSELIN2.bit.INT1CONT = 0; //Disable ADCINT1 Continuous mode
    AdcRegs.INTSELIN2.bit.INT1SEL = 1; //setup EOC1 to trigger ADCINT1 to fire
    AdcRegs.ADCSOC0CTL.bit.CHSEL = 7; //set SOC0 channel select to ADCINA7 J5.3
    AdcRegs.ADCSOC1CTL.bit.CHSEL = 3; //set SOC1 channel select to ADCINA3 J5.4
    AdcRegs.ADCSOC0CTL.bit.TRIGSEL = 7; //set SOC0 start trigger on EPWM2A, due to round-robin SOC0 converts first then SOC1
    AdcRegs.ADCSOC1CTL.bit.TRIGSEL = 7;
    AdcRegs.ADCSOC0CTL.bit.ACQPS = 15; //set SOC0 S/H Window to 15 ADC Clock Cycles, (14 ACQPS plus 1)
    AdcRegs.ADCSOC1CTL.bit.ACQPS = 15; //set SOC1 S/H Window to 15 ADC Clock Cycles, (14 ACQPS plus 1)
    EDIS;
}

// PWM setup with PWM outputs on J6.1 and J6.2 of the F28027 LaunchPad
// Reference: Sec. 3.2.2 Page 215 of https://www.ti.com/lit/ug/sprui09/sprui09.pdf
void InitEPwm1Example(unsigned int period, unsigned int pulse_width)
{
    // Setup TBCLK
    EPwm1Regs.TBPRD = period-1; // TBPRD = (CLOCK_FREQ/PWM_FREQ)-1
    EPwm1Regs.TBPHS.half.TBPHS = 0x0000; // Phase is 0. This register determines the phase between previous EPWM and the current EPWM
pin.
    EPwm1Regs.TBCTR = 0x0000; // Clear counter for trailing-edge PWM.

    // Set Compare values
    EPwm1Regs.CMPA.half.CMPA = pulse_width; // This sets the initial duty cycle on EPWM1A or J6.1. CMPA = (pulse_width)/(TBPRD+1)
    EPwm1Regs.CMPB = pulse_width; // This sets the initial duty cycle on EPWM1B or J6.2. CMPB = (pulse_width)/(TBPRD+1)

    // Setup counter mode
    EPwm1Regs.TBCTL.bit.CTRMODE = TB_COUNT_UP; // This sets the PWM type, ie trailing, leading or symmetric.
    EPwm1Regs.TBCTL.bit.PHSEN = TB_ENABLE; // Disable phase loading
    EPwm1Regs.TBCTL.bit.HSPCLKDIV = TB_DIV1; // Clock ratio to SYSCLKOUT
    EPwm1Regs.TBCTL.bit.CLKDIV = TB_DIV1; // PWM clock prescale

    // Setup shadowing
    EPwm1Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
    EPwm1Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
    EPwm1Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // Load on Zero
    EPwm1Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO;

    // Set actions
    EPwm1Regs.AQCTLA.bit.ZRO = AQ_SET; // Sets PWM1A J6.1 when period starts
    EPwm1Regs.AQCTLA.bit.CAU = AQ_CLEAR; // Clears PWM1A J6.1 when CMPA > sawtooth

    EPwm1Regs.AQCTLB.bit.ZRO = AQ_SET; // Sets PWM1B J6.2 when period starts
    EPwm1Regs.AQCTLB.bit.CBU = AQ_CLEAR; // Clears PWM1B J6.2 when CMPB > sawtooth

    ///////////////EPWM2 FOR ADC SOC////////////////////
    // Setup TBCLK
    EPwm2Regs.TBPRD = period-1; // TBPRD = (CLOCK_FREQ/PWM_FREQ)-1
    EPwm2Regs.TBPHS.half.TBPHS = 0x0000; // Phase is 0. This register determines the phase between previous EPWM and the current EPWM
pin.
    EPwm2Regs.TBCTR = 0x0000; // Clear counter for trailing-edge PWM.

    // Set Compare values
    EPwm2Regs.CMPA.half.CMPA = pulse_width; // This sets the initial duty cycle on EPWM2A or J6.3. CMPA = (pulse_width)/(TBPRD+1)
    EPwm2Regs.CMPB = pulse_width; // This sets the initial duty cycle on EPWM2B or J6.4. CMPB = (pulse_width)/(TBPRD+1)

    // Setup counter mode
    EPwm2Regs.TBCTL.bit.CTRMODE = TB_COUNT_UP; // This sets the PWM type, ie trailing, leading or symmetric.
    EPwm2Regs.TBCTL.bit.PHSEN = TB_ENABLE; // Disable phase loading
    EPwm2Regs.TBCTL.bit.HSPCLKDIV = TB_DIV1; // Clock ratio to SYSCLKOUT
    EPwm2Regs.TBCTL.bit.CLKDIV = TB_DIV1; // PWM clock prescale

    // Setup shadowing
    EPwm2Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
    EPwm2Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
    EPwm2Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // Load on Zero
    EPwm2Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO;

    // Set actions
    EPwm2Regs.AQCTLA.bit.ZRO = AQ_SET; // Sets PWM1A J6.3 when period starts
    EPwm2Regs.AQCTLA.bit.CAU = AQ_CLEAR; // Clears PWM1A J6.4 when CMPA > sawtooth

    EPwm2Regs.AQCTLB.bit.ZRO = AQ_SET; // Sets PWM1B J6.3 when period starts
    EPwm2Regs.AQCTLB.bit.CBU = AQ_CLEAR; // Clears PWM1B J6.4 when CMPB > sawtooth

    // ADC synchronization with EPWM 2A, i.e. PWM output on pin J6.3
    EPwm2Regs.ETSEL.bit.SOCAEN = 1; // Enable SOC on A group
    EPwm2Regs.ETSEL.bit.SOCASEL = 4; // Select SOC from CMPA on upcount
    EPwm2Regs.ETPS.bit.SOCAPRD = 1; // Generate pulse on 1st event

```

```

}

void InitEpwm3_4(unsigned int mod_period, unsigned int half_duty)
{
    // EPWM 3 SETUP
    // Setup TBCLK
    EPwm3Regs.TBPRD = mod_period-1; // TBPRD = (TIME_BASE_CLOCK_FREQ/PWM_FREQ)-1. Check line # 243 in the code for
    TIME_BASE_CLOCK_FREQ setup
    EPwm3Regs.TBPHS.half.TBPHS = 0x0000; // Phase is 0. This register determines the phase between previous EPWM and the current EPWM
    pin.
    EPwm3Regs.TBCTL.bit.SYNCSEL = TB_CTR_ZERO; // Sync down-stream module
    EPwm3Regs.TBCTR = 0x0000; // Clear counter for trailing-edge PWM.

    // Set Compare values
    EPwm3Regs.CMPA.half.CMPA = half_duty; // This sets the initial duty cycle of 50% on EPWM1A or J6.5. CMPA =
    (pulse_width)/(TBPRD+1)
    EPwm3Regs.CMPB = half_duty; // This sets the initial duty cycle of 50% on EPWM1B or J6.6. CMPB =
    (pulse_width)/(TBPRD+1)

    // Setup counter mode
    EPwm3Regs.TBCTL.bit.CTRMODE = TB_COUNT_UP; // This sets the PWM type, ie trailing, leading or symmetric.
    EPwm3Regs.TBCTL.bit.PHSEN = TB_ENABLE; // Disable phase loading
    EPwm3Regs.TBCTL.bit.HSPCLKDIV = TB_DIV1; // Clock ratio to SYSCLKOUT
    EPwm3Regs.TBCTL.bit.CLKDIV = TB_DIV1; // PWM clock prescale

    // Setup shadowing
    EPwm3Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
    EPwm3Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
    EPwm3Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // Load on Zero
    EPwm3Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO;

    // Set actions
    EPwm3Regs.AQCTLA.bit.ZRO = AQ_SET; // Sets PWM1A J6.5 when period starts
    EPwm3Regs.AQCTLA.bit.CAU = AQ_CLEAR; // Clears PWM1A J6.5 when CMPA > sawtooth

    EPwm3Regs.AQCTLB.bit.ZRO = AQ_CLEAR; // Sets PWM1B J6.6 when period starts
    EPwm3Regs.AQCTLB.bit.CBU = AQ_SET; // Clears PWM1B J6.6 when CMPB > sawtooth

    // // Set Deadband
    // EPwm3Regs.DBCTL.bit.OUT_MODE = DB_FULL_ENABLE; // Enable rising and falling edge deadband
    // EPwm3Regs.DBCTL.bit.POLSEL = DB_ACTV_HIC; // Sets the EPWMA pin as a source for deadband on EPWMB's rising and falling edge. This
    // is also compliments the EPWMA pin and sets that as output on EPWMB
    // EPwm3Regs.DBCTL.bit.IN_MODE = DBA_ALL;
    // EPwm3Regs.DBRED = 20; // Sets a rising edge deadband on EPWMA. DBRED = (rising edge time / Time base clock)
    // EPwm3Regs.DBFED = 20; // Sets a falling edge deadband on EPWMB. DBFED = (falling edge time / Time base
    // clock)

    // EPWM 4 SETUP
    // Setup TBCLK
    EPwm4Regs.TBPRD = mod_period; // TBPRD = (TIME_BASE_CLOCK_FREQ/PWM_FREQ)-1. Check line # 280 in the code for
    TIME_BASE_CLOCK_FREQ setup
    EPwm4Regs.TBPHS.half.TBPHS = delay_shift; // Phase is 180 degrees. This register determines the phase between previous EPWM and the
    current EPWM pin.
    EPwm4Regs.TBCTL.bit.SYNCSEL = TB_SYNC_IN; // Syncs the counter of EPWM 4 with EPWM 3
    EPwm4Regs.TBCTR = 0x0000; // Clear counter for trailing-edge PWM.

    // Set Compare values
    EPwm4Regs.CMPA.half.CMPA = half_duty; // This sets the initial duty cycle on EPWM1A or J2.8. CMPA = (pulse_width)/(TBPRD+1)
    EPwm4Regs.CMPB = half_duty; // This sets the initial duty cycle on EPWM1B or J2.9. CMPB = (pulse_width)/(TBPRD+1)

    // Setup counter mode
    EPwm4Regs.TBCTL.bit.CTRMODE = TB_COUNT_UP; // This sets the PWM type, ie trailing, leading or symmetric.
    EPwm4Regs.TBCTL.bit.PHSEN = TB_ENABLE; // Enable phase loading sets the phase of EPWM 4 with respect to EPWM 3
    EPwm4Regs.TBCTL.bit.HSPCLKDIV = 0; // Clock ratio to SYSCLKOUT
    EPwm4Regs.TBCTL.bit.CLKDIV = 4; // PWM clock prescale set to 8. Timebase clock frequency = (60 MHz / 8) = 7.5 MHz

    // Setup shadowing
    EPwm4Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
    EPwm4Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
    EPwm4Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // Load on Zero
    EPwm4Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO;

    // Set actions
    EPwm4Regs.AQCTLA.bit.ZRO = AQ_SET; // Sets PWM1A J2.8 when period starts
    EPwm4Regs.AQCTLA.bit.CAU = AQ_CLEAR; // Clears PWM1A J2.8 when CMPA > sawtooth

    EPwm4Regs.AQCTLB.bit.ZRO = AQ_SET; // Sets PWM1B J2.9 when period starts
    EPwm4Regs.AQCTLB.bit.CBU = AQ_CLEAR; // Clears PWM1B J2.9 when CMPB > sawtooth

    // // Set Deadband
    // EPwm4Regs.DBCTL.bit.OUT_MODE = DB_FULL_ENABLE; // Enable rising and falling edge deadband
    // EPwm4Regs.DBCTL.bit.POLSEL = DB_ACTV_HIC; // Sets the EPWMA pin as a source for deadband on EPWMB's rising and falling edge. This
    // is also compliments the EPWMA pin and sets that as output on EPWMB
    // EPwm4Regs.DBCTL.bit.IN_MODE = DBA_ALL;
    // EPwm4Regs.DBRED = 20; // Sets a rising edge deadband on EPWMA. DBRED = (rising edge time / Time base clock)
    // EPwm4Regs.DBFED = 20; // Sets a falling edge deadband on EPWMB. DBFED = (falling edge time / Time base clock)
}

```

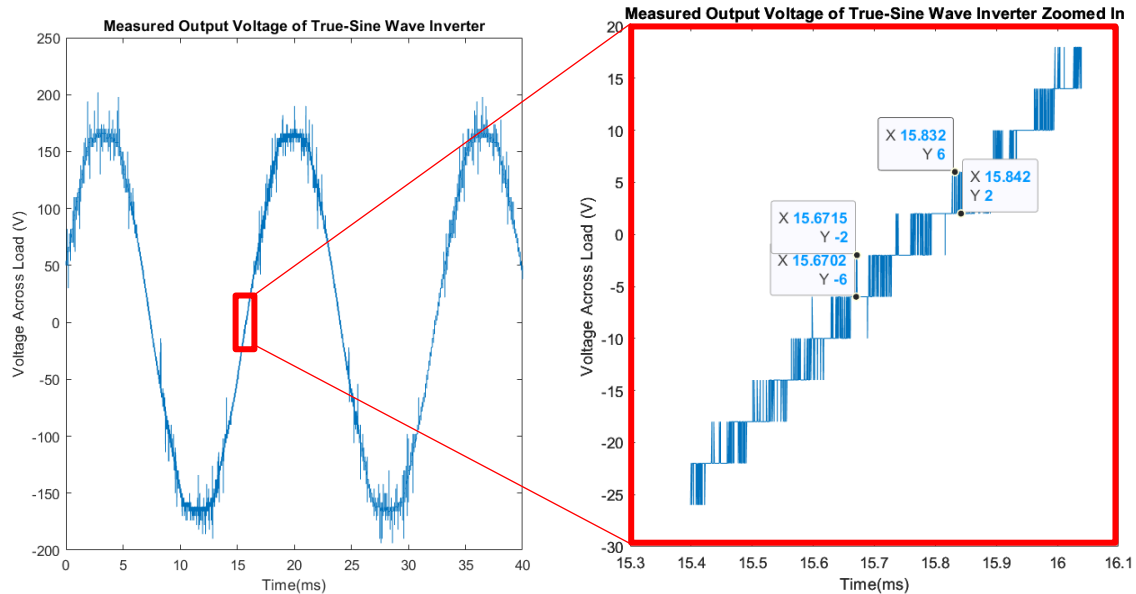
iii. Measured Input DC Voltage, Output RMS Voltage, Output RMS Current, Output Voltage Ripple:

Typically, the output voltage ripple is greatest when the sinusoid is changing fastest. We will assume the ripple measured at the peaks is just noise associated with duty-cycles at relatively extreme values and measure the ripple at the point of fastest slope instead.

Measured Input Voltage	DC	Output RMS Voltage	Output RMS Current	Output Voltage Ripple
193.8V		122.1V	0.3A	4V

Measured frequency was **60.19 Hz**.

iv. Measured Output Voltage Waveform:



The resolution with which the first scope-capture was taken unfortunately did not give too much resolution, but enough nonetheless to show that the ripple never exceeds 4V in the sinusoidal region where duty cycle changes the fastest.